```
class SomeClass
// ...
```

... where `SomeAnnotation` would be a class provided by the library supporting the annotation and propped up by code that will go and read your source code files. Most major frameworks and libraries (Symfony, Laravel, PHPUnit, etc.) used comment annotations and migrated to attributes on the release of PHP 8.

Making Attributes

To create your own attribute, you simply need to mark a class with `#[Attribute]`, which immediately makes it available for use within the rest of your code. (The attribute class still needs to follow the same rules as regular classes for autoloading and namespacing, so if your attribute is in a different namespace, then import it with `use`.) Let's try it out:

```
#[Attribute]
class Healer
{
}

#[Attribute]
class Fighter
{
}
```

The attribute classes `Fighter` and `Healer` don't actually do anything, but even empty classes can be useful in a system like this. In our case they're going to serve simply as meta-properties for other classes, which we can test when those marked classes are used at runtime. Let's mark some classes with these attributes:

```
#[Healer]
class Cleric
{
}

#[Fighter]
class Barbarian
{
}
```